23

Hotel Reservation System

Design a hotel reservation system. Users can search for available rooms and make reservations. Hotel staff can view and manage reservations.

Step 1 - Understand the Problem and Establish Design Scope

Some of the questions we should ask the interviewer include:

Candidate: Is this a website or a mobile app?
Interviewer: Both.
Candidate: Do customers need to pay for the reservation, or only make a reservation?
Interviewer: Customers can make a reservation without payment.
Candidate: How many hotels do we need to support?
Interviewer: Assume there are 1000 hotels in the system.
Candidate: How many rooms do a typical hotel have?
Interviewer: Assume a typical hotel has 100 rooms.
Candidate: How far in advance can people make a reservation?
Interviewer: Assume people can make a reservation up to one year in advance.
Candidate: Do customers need to be able to cancel a reservation?
Interviewer: Yes.
Candidate: Do customers need to be able to modify a reservation?
Interviewer: Yes.
Candidate: Do customers need to be able to see the availability of a hotel room before they make a reservation?
Interviewer: Yes.
Candidate: Do we need to support payment processing?
Interviewer: Payment processing is out of scope for this interview.

Some other things we should clarify with our interviewer include:

- Do customers need to be notified when they make a reservation?
- Do we need to support overbooking?
- Do we need to support waitlists?
- Do we need to support hotel staff?

Non-functional requirements

- Strong consistency for reservations to avoid double bookings.
- Low latency for room availability searches.
- Support 1000 hotels and 100 rooms per hotel.
- Support up to one year in advance bookings.

Out of the scope

- Payment processing.
- Overbooking.
- Waitlists.
- Hotel staff management.

Step 2 - Propose High-Level Design and Get Buy-In

Actors and Actions

- Guest
- Hotel Staff
- System

API design

Guest

GET	/hotels/{hotelId}/rooms/search?checkIn=...&checkOut=...	Search available rooms
POST	/reservations	Create a reservation
PUT	/reservations/{reservationId}	Modify a reservation
DELETE	/reservations/{reservationId}	Cancel a reservation

Hotel Staff

GET	/hotels/{hotelId}/reservations?date=...	View reservations for a hotel
GET	/reservations/{reservationId}	View reservation details
PUT	/reservations/{reservationId}/status	Update reservation status

Request parameters

GET	/hotels/{hotelId}/rooms/search?checkIn=...&checkOut=...	Search available rooms
	checkIn	Check-in date (YYYY-MM-DD)
	checkOut	Check-out date (YYYY-MM-DD)
POST	/reservations	Create a reservation
	hotelId	Hotel ID
	roomTypeId	Room type ID
	checkIn	Check-in date (YYYY-MM-DD)
	checkOut	Check-out date (YYYY-MM-DD)
	guestCount	Number of guests
	guestInfo	Guest name, email, phone
PUT	/reservations/{reservationId}	Modify a reservation
	checkIn	New check-in date (optional)
	checkOut	New check-out date (optional)
	guestCount	New guest count (optional)
DELETE	/reservations/{reservationId}	Cancel a reservation
GET	/hotels/{hotelId}/reservations?date=...	View reservations for a hotel
	hotelId	Hotel ID
	date	Date (YYYY-MM-DD)
GET	/reservations/{reservationId}	View reservation details
	reservationId	Reservation ID
PUT	/reservations/{reservationId}/status	Update reservation status
	reservationId	Reservation ID
	status	New status (confirmed, cancelled, checked_in, checked_out)

Response codes

GET	/hotels/{hotelId}/rooms/search?checkIn=...&checkOut=...	Search available rooms
	200 OK	Success, return available rooms
	400 Bad Request	Invalid date format or missing parameters
POST	/reservations	Create a reservation
	201 Created	Reservation created successfully
	400 Bad Request	Invalid input or missing required fields
	409 Conflict	Room unavailable for requested dates
PUT	/reservations/{reservationId}	Modify a reservation
	200 OK	Reservation updated successfully
	400 Bad Request	Invalid input
	404 Not Found	Reservation not found
	409 Conflict	Room unavailable for requested dates
DELETE	/reservations/{reservationId}	Cancel a reservation
	204 No Content	Reservation cancelled successfully
	404 Not Found	Reservation not found
GET	/hotels/{hotelId}/reservations?date=...	View reservations for a hotel
	200 OK	Success, return reservations
	400 Bad Request	Invalid date format
GET	/reservations/{reservationId}	View reservation details
	200 OK	Success, return reservation details
	404 Not Found	Reservation not found
PUT	/reservations/{reservationId}/status	Update reservation status
	200 OK	Status updated successfully
	400 Bad Request	Invalid status
	404 Not Found	Reservation not found

Note: Error response format

```
{
  "error": "Room unavailable for requested dates"
}
```

Data model

Based on the API design, we can see that we need to store hotels, rooms, and reservations. We will also need a table for guest information.

- Hotels: hotelId, name, address, timezone
- Rooms: roomId, hotelId, roomTypeId, roomNumber
- Reservations: reservationId, hotelId, roomId, guestId, checkIn, checkOut, guestCount, status, totalPrice, specialRequests, cancellationReason, cancelledAt, checkedInAt, checkedOutAt, createdAt, updatedAt
- Guests: guestId, name, email, phone

High-level design

Figure 1: High-level design

We need a load balancer to distribute traffic across multiple application servers. The application servers handle all business logic and connect to the database. We use a single database for now, which we can scale later if needed.

Deep Dive: Room availability

The core challenge is determining which rooms are available for a given date range. We need to check for overlapping reservations and ensure we can handle concurrent requests without double bookings.

We need to store room availability for each date. For a room with 100 rooms and 365 days, we need 36,500 records per room type. For 1000 hotels, this gives us 36 million records, which is manageable.

We can use two approaches:

- Pre-calculated availability: Store availability in a separate table. Fast queries but requires maintenance.
- Real-time calculation: Calculate availability from reservations. Slower but always accurate.

We choose real-time calculation for simplicity and accuracy. We can optimize later if needed.

To prevent double bookings, we need strong consistency. We use database transactions with SERIALIZABLE isolation level. When a user makes a reservation, we check availability and create the reservation in a single transaction. If another user tries to book the same room at the same time, one transaction will fail and retry.

Figure 2: Deep dive: Room availability

Deep Dive: Concurrent reservations

When multiple users try to book the same room at the same time, we need to prevent double bookings. We use SERIALIZABLE isolation level for the reservation transaction. When two transactions conflict, one will fail and retry. The application server handles retry logic with exponential backoff.

We also add a unique constraint on (roomId, checkIn, checkOut) to prevent exact duplicate reservations at the database level.

Figure 3: Deep dive: Concurrent reservations

Step 3 - Design Deep Dives

Proposed data model

- Hotels table
- Rooms table
- Reservations table
- Guests table

Reservations table

Figure 4: Reservations table

Guests table

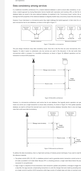

Figure 5: Guests table

Indexes

- idx_reservations_hotelId_checkIn_checkOut
- idx_reservations_guestId
- idx_reservations_status
- idx_rooms_hotelId_roomTypeId
- idx_guests_email

Scaling

Read operations dominate (room availability searches). We can scale read capacity by adding read replicas. Write operations (reservations) need to go to the primary database.

We can shard by hotelId if we need to scale beyond one database. Each hotel's data stays on one shard, so we can handle transactions within a single shard.

Figure 6: Scaling

Step 4 - Wrap Up

In this design, we built a hotel reservation system that prevents double bookings using database transactions with SERIALIZABLE isolation. We use real-time calculation for room availability to ensure accuracy. The system can scale read operations by adding replicas and write operations by sharding by hotelId.

Chapter Summary

Reference Material

- Hotel Reservation System - Hello Interview
- Hotel Reservation System - Tech Prep
- Hotel Reservation System - Tech Prep
- Hotel Reservation System - Tech Prep